

2^ο ΙΕΚ ΗΡΑΚΛΕΙΟΥ

ΜΑΘΗΜΑ: ΑΝΤΙΚΕΙΜΕΝΟΣΤΡΑΦΗΣ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ (C++)

Φύλλο Ανάθεσης Εργασίας Αριθμ. 20

Πολυμορφισμός – Virtual methods

Μια σημαντική έννοια του Αντικειμενοστραφούς Προγραμματισμού είναι ο πολυμορφισμός, που παρατηρείται σε μεθόδους που έχουν οριστεί με διαφορετικό τρόπο σε κλάσεις που ανήκουν σε μια ιεραρχία.

Δραστηριότητα 1 – Ορισμός πολυμορφικών μεθόδων

Δημιουργήστε ένα αρχείο με όνομα polymorphism1.cpp με το παρακάτω περιεχόμενο:

```
#include <iostream>
using namespace std;

class Shape {
protected:
    int width, height;
public:
    Shape( int a = 0, int b = 0){
        width = a;
        height = b;
    }

    void print() {
        cout << "Shape object" << endl;
        cout << "Area = " << area() << endl;
    }

    int area() {
        return 0;
    };
};

class Rectangle: public Shape {
public:
    Rectangle( int a = 0, int b = 0):Shape(a, b) { }

    int area() {
        return width*height;
    };

    void print() {
        cout << "Rectangle object" << endl;
        cout << "Area = " << area() << endl;
    }
};

class Triangle: public Shape {
public:
    Triangle( int a = 0, int b = 0):Shape(a, b) { }

    int area() {
        return (width*height)/2;
    };

    void print() {
        cout << "Trangle object " << endl;
        cout << "Area = " << area() << endl;
    }
};
```

```

int main() {
    Shape *shape;
    Rectangle rec(10,7);
    Triangle tri(10,5);
    shape = &rec; // store the address of Rectangle
    shape->print(); // call rectangle area.
    shape = &tri; // store the address of Triangle
    shape->print(); // call triangle area.
    return 0;
}

```

Στο παραπάνω παράδειγμα ορίζουμε διαφορετικές υλοποιήσεις της μεθόδου `area()` στη βασική κλάση (`Shape`) από τις κλάσεις `Rectangle` και `Triangle` που κληρονομούν από τη `Shape`. Παρατηρήστε ότι όταν καλούμε τη μέθοδο `area()`, μέσω του δείκτη `shape`, εκτελείται η μέθοδος `area()` της κλάσης `Shape`, ανεξάρτητα από το αντικείμενο στο οποίο δείχνει ο δείκτης `shape` (`Rectangle` ή `Triangle`).

Δραστηριότητα 2 – Ορισμός `virtual` μεθόδου

Δημιουργήστε ένα αρχείο με όνομα `polymorphism2.cpp` με το παρακάτω περιεχόμενο, όπου η μόνη αλλαγή είναι η προσθήκη της λέξης `virtual` στον ορισμό των μεθόδων `print()` και `area()` στην κλάση `Shape` και εκτελέστε το:

```

#include <iostream>
using namespace std;

class Shape {
protected:
    int width, height;
public:
    Shape( int a = 0, int b = 0){
        width = a;
        height = b;
    }

    virtual void print() {
        cout << "Shape object" << endl;
        cout << "Area = " << area() << endl;
    }

    virtual int area() {
        return 0;
    };
};

class Rectangle: public Shape {
public:
    Rectangle( int a = 0, int b = 0):Shape(a, b) { }

    int area() {
        return width*height;
    };

    void print() {
        cout << "Rectangle object" << endl;
        cout << "Area = " << area() << endl;
    }
};

class Triangle: public Shape {
public:

```

```

Triangle( int a = 0, int b = 0):Shape(a, b) { }

    int area() {
        return (width*height)/2;
    };

    void print() {
        cout << "Trangle object " << endl;
        cout << "Area = " << area() << endl;
    }

};

int main() {
    Shape *shape;
    Rectangle rec(10,7);
    Triangle tri(10,5);
    shape = &rec; // store the address of Rectangle
    shape->print(); // call rectangle area.
    shape = &tri; // store the address of Triangle
    shape->print(); // call triangle area.
    return 0;
}

```

Παρατηρήστε ότι πλέον όταν καλούμε τη μέθοδο `area()`, μέσω του δείκτη `shape`, εκτελείται η μέθοδος `area()` της κλάσης `Rectangle` και `Triangle` κατά περίπτωση, ανάλογα με το αντικείμενο που δείχνει ο δείκτης `shape`. Αυτό οφείλεται στην ιδιότητα του πολυμορφισμού, όπου ο μεταγλωττιστής γνωρίζει τότε πρέπει να καλέσει ποια μέθοδο, βασισμένος στον τύπο του τρέχοντος αντικειμένου.

Δραστηριότητα 3 – Ορισμός γνήσιας `virtual` μεθόδου

Όταν ορίζουμε γνήσια `virtual` μέθοδο σε μια κλάση τότε δηλώνουμε ότι δε θα υπάρχει υλοποίηση στην κλάση αυτή, αλλά στις υπο-κλάσεις της.

Δημιουργήστε ένα αρχείο με όνομα `polymorphism3.cpp` με το παρακάτω περιεχόμενο, όπου η μόνη αλλαγή είναι η υλοποίηση της `virtual` μεθόδου `area()` στην κλάση `Shape`, και εκτελέστε το:

```

#include <iostream>
using namespace std;

class Shape {
protected:
    int width, height;
public:
    Shape( int a = 0, int b = 0){
        width = a;
        height = b;
    }

    virtual void print() = 0;

    virtual int area() = 0;
};

class Rectangle: public Shape {
public:
    Rectangle( int a = 0, int b = 0):Shape(a, b) { }

    int area() {
        return width*height;
    }
}

```

```

};

void print() {
    cout << "Rectangle object" << endl;
    cout << "Area = " << area() << endl;
}

};

class Triangle: public Shape {
public:
    Triangle( int a = 0, int b = 0):Shape(a, b) { }

    int area() {
        return (width*height)/2;
    };

    void print() {
        cout << "Trangle object " << endl;
        cout << "Area = " << area() << endl;
    }

};

int main() {
    Shape *shape;
    Rectangle rec(10,7);
    Triangle tri(10,5);
    shape = &rec; // store the address of Rectangle
    shape->print(); // call rectangle area.
    shape = &tri; // store the address of Triangle
    shape->print(); // call triangle area.
    return 0;
}

```